

Modelling Software-based Systems

Lecture 6

Validation, Verification and Proof Tools for Event-B for Correct-by-Construction

Telecom Nancy (IL et LE)

Dominique Méry
Telecom Nancy, Université de Lorraine

7 septembre 2026
dominique.mery@loria.fr

- 1 Verification
- 2 Proof Management
- 3 Development Methodology and Tools for Correct By Construction
- 4 Project Management

- 1 Verification
- 2 Proof Management
- 3 Development Methodology and Tools for Correct By Construction
- 4 Project Management

$$\begin{array}{l} \ell : \{P_\ell(v)\} \\ \text{cond}_{\ell, \ell'}(v) \longrightarrow v := f_{\ell, \ell'}(v) \\ \ell' : \{P_{\ell'}(v)\} \end{array}$$
$$\begin{array}{l} e(\ell, \ell') \\ \textbf{WHEN} \\ \quad c := \ell \\ \quad \text{cond}_{\ell, \ell'}(v) \\ \textbf{THEN} \\ \quad c := \ell' \\ \quad v := f_{\ell, \ell'}(v) \\ \textbf{END} \end{array}$$
$$\begin{array}{l} \ell_0^1 : \{x = 0\} \\ \quad x := x + 1; \\ \ell_0^1 : \{x = 1\} \end{array}$$

- v is the state memory variable or list of memory variables; v includes the local variables and the results variables.
- c is a new variable which is modelling the control flow and its type is **LOCATIONS**.
- $e(\ell, \ell')$ is simulating the computation flow starting from ℓ and moving to ℓ' ; v is updated.

INVARIANTS

$inv_i : c \in \text{LOCATIONS}$

$inv_j : v \in \text{Type}$

...

$inv_k : c = \ell \Rightarrow P_\ell(v)$

$inv_m : c = \ell' \Rightarrow P_{\ell'}(v)$

...

$th_n : A(c, v)$

- Type is the type of the variables v and is a set of possible values defined in the context C .
- The annotation is giving us for free the conditions satisfied by v when the control is in ℓ , (resp. in ℓ').
- $A(c, v)$ is a safety property that we are supposed to check and the case of Event-B, it is a theorem.

For each pair of successive labels ℓ, ℓ' , the three statements are equivalent :

- $P_\ell(v) \wedge \text{cond}_{\ell, \ell'}(v) \wedge v' = f_{\ell, \ell'}(v) \Rightarrow P_{\ell'}(v')$
- $I(c, v) \wedge c = \ell \wedge \text{cond}_{\ell, \ell'}(v) \wedge c' = \ell' \wedge v' = f_{\ell, \ell'}(v) \Rightarrow (c' = \ell' \Rightarrow P_{\ell'}(v'))$
- $I(c, v) \wedge BA(e(\ell, \ell'))(c, v, c', v') \Rightarrow (c' = \ell' \Rightarrow P_{\ell'}(v'))$

L

Let AA an annotated algorithm with precondition $\mathbf{pre}(AA)(v)$ and postcondition $\mathbf{post}(AA)(v_0, v)$. Let the context C and the machine M generated from AA using the construction given previously. We assume that ℓ_0 is the first label and ℓ_e is the last label. We add the following safety properties in the machine M :

- $c = \ell_0 \wedge \mathbf{pre}(AA)(v) \Rightarrow P_{\ell_0}(v)$
- $c = \ell_e \Rightarrow (P_{\ell_e}(v) \Rightarrow \mathbf{post}(AA)(v_0, v))$

If proof obligations are discharged, then the annotated algorithm AA is partially correct with respect to its pre/post specification.

- 1 Verification
- 2 **Proof Management**
- 3 Development Methodology and Tools for Correct By Construction
- 4 Project Management

- The panel called *proof control* is providing tools for *discharging* proof obligations
- Proof obligations are stated as follows : $\Gamma \vdash P$: *When assertions of Γ are supposed to hold, then P does hold*
- Problem : Checking that the sequent $\Gamma \vdash P$ is valid.
- How : Applying automatic procedures as pp, pr etc
- ... or helping the proof tool to build the *proof tree*

- Quite sensitive to **useless hypotheses**.
- Unable to **automatically abstract** some statements.
- Sometimes **poor management of equalities**.
- Needs some **creative hints**

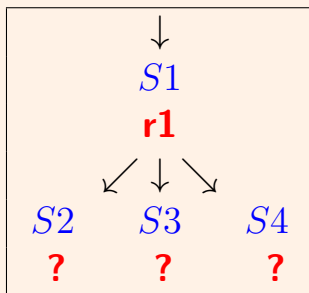
- **Sequent** (where *list_of_hypotheses* might be empty).

$$\textit{list_of_hypotheses} \vdash \textit{conclusion}$$

- **Inference rule** (where *list_of_sequents* might be empty).

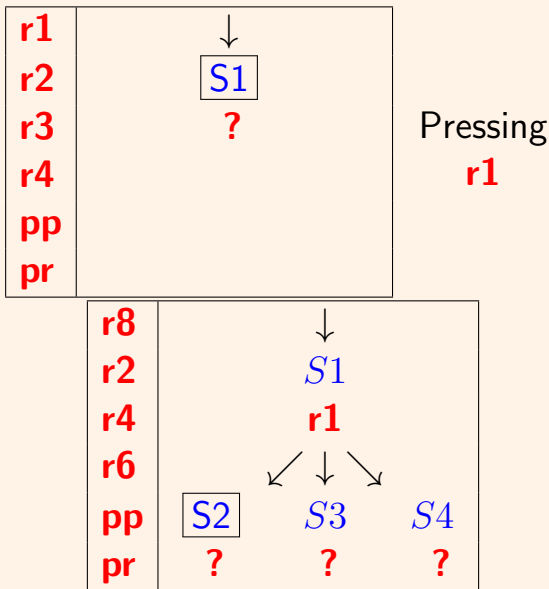
$$\frac{\textit{list_of_sequents}}{\textit{sequent}}$$

- **Backwards application** of inf. rules $\leadsto$ **proof tree**.



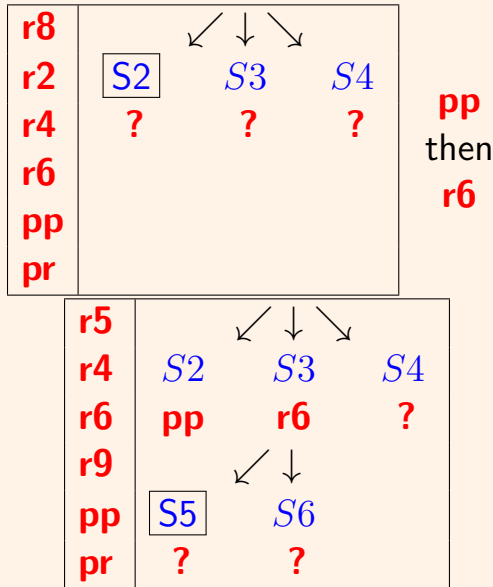
- Rule $r1$ is applied to sequent $S1$.
- Producing sequents $S2$, $S3$, and $S4$.

Interface : Tree and Palette of Rule Buttons



- The proof tree is **constructed interactively**.

Difficulty : Size of Window



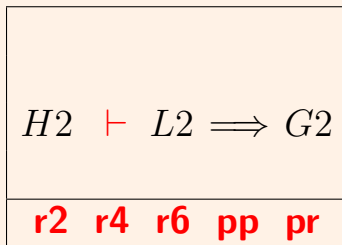
- Our tentative “interface” does not work.

$$H \vdash L \Longrightarrow G$$

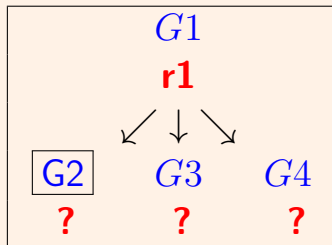
- Most of the time the conclusion has an **implicative form**
- G is called the **goal** (it is a **non-conjunctive** predicate)
- L are called the **local hypotheses** (might be empty)

$$H \vdash I(x) \wedge G(x) \wedge P(x, x') \Longrightarrow I(x')$$

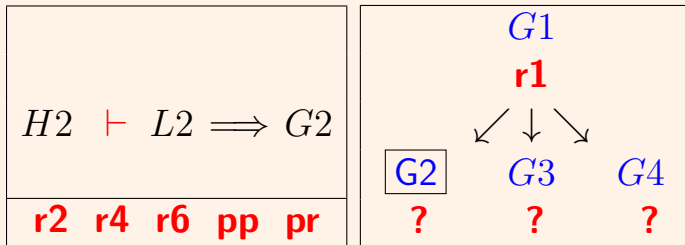
- **Tree window** with **simplified** sequents (**goals** only)
- **Sequent window** containing the **full sequent of interest**



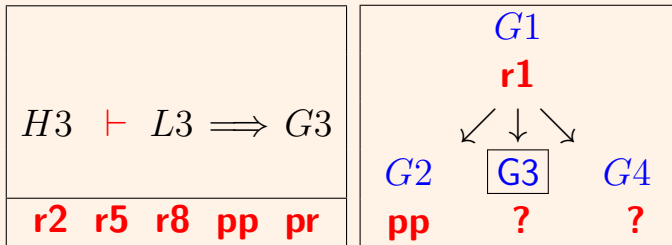
Sequent Window

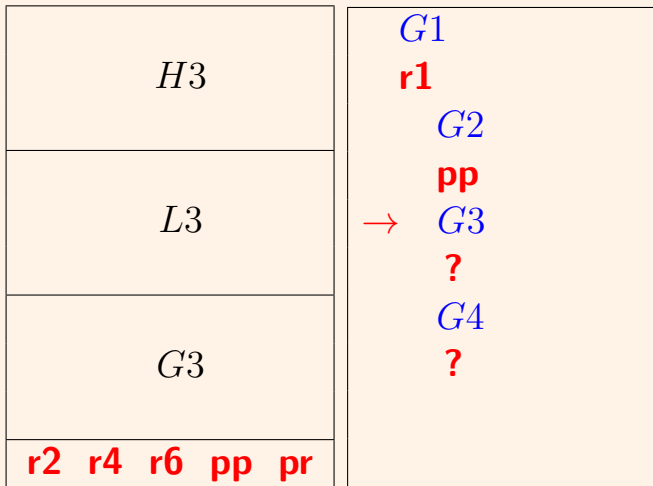


Tree Window



Pressing **pp**





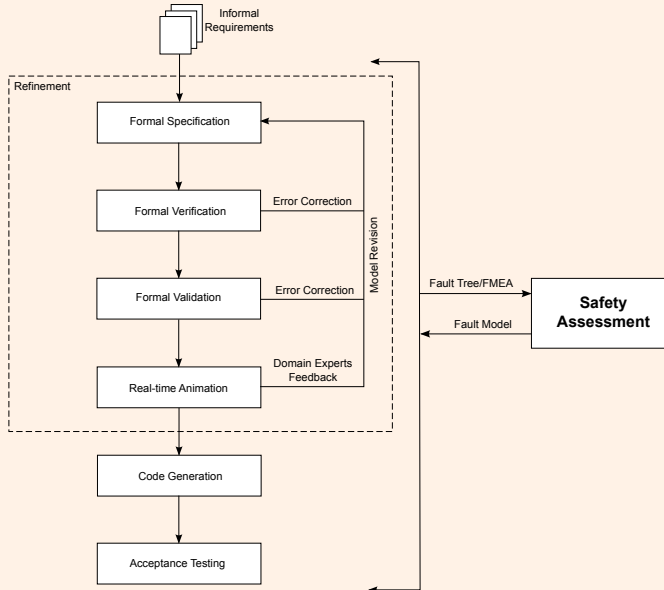
$$\underbrace{hidden ; searched ; cached}_{list_of_hypotheses} \vdash \underbrace{local \implies goal}_{conclusion}$$

$hidden$	Not visible on the sequent window
$searched$	Visible after some search in $hidden$
$cached$	Visible but not part of the conclusion any m
$local$	Visible and part of the conclusion

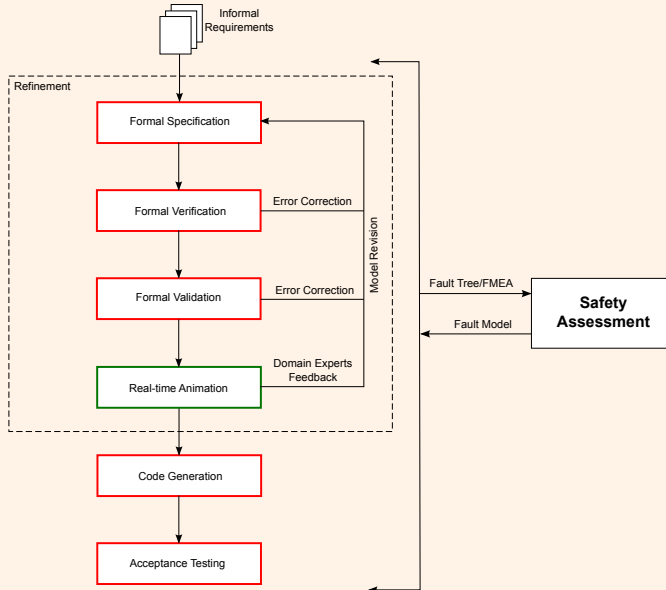
Current Summary

- 1 Verification
- 2 Proof Management
- 3 Development Methodology and Tools for Correct By Construction
- 4 Project Management

Critical System Development Life-Cycle Methodology



Critical System Development Life-Cycle Methodology



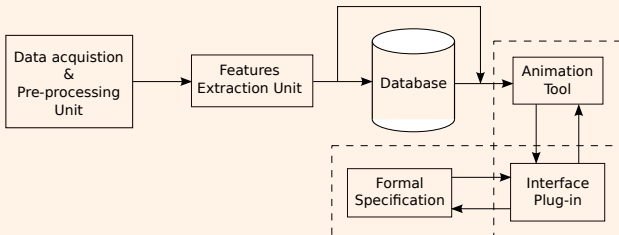
What is Real-time Animator ?

- Visual representation of formal model using real time data set.

Why should we use Formal Model Animator ?

- To validate system behavior according to the stakeholders
- To express formal models for non-mathematical domain experts
- To discover the error in the early stage of system development (Traceability)

Proposed Architecture

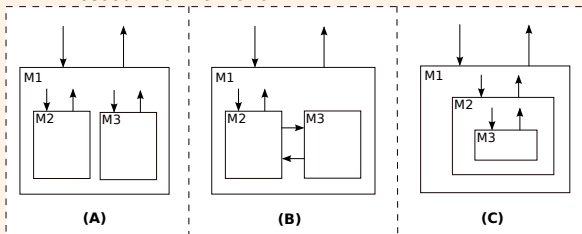


- Refinement chart is a graphical representation of a complex system using layering approach.
- Integration and features based code structuring of different components of the systems.

Basic rules of refinement chart

- Parallel Refinement
- Sequential Refinement
- Nested Refinement

$$\begin{aligned} M1 &\sqsubseteq (M2 \parallel M3 \parallel \dots \parallel M_{n-1} \parallel M_n) \\ M1 &\sqsubseteq (M2 \succ M3 \succ \dots \succ M_{n-1} \succ M_n) \\ (M1 &\sqsubseteq M2, M2 \sqsubseteq M3, \dots, M_{n-1} \sqsubseteq M_n) \end{aligned}$$

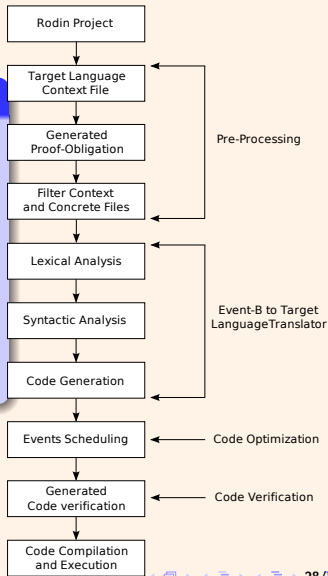


$$\begin{aligned} M1 &\sqsubseteq (M2 \parallel M3) \\ M1 &\sqsubseteq (M2 \succ M3) \\ (M1 &\sqsubseteq M2, M2 \sqsubseteq M3) \end{aligned}$$

EB2ALL

- Automatic code generation from Event-B formal specification to multiple languages (C, C++, Java, C#) to make it executable.
- A module for Rodin (Deploy is an European Commission FP7 project)
- It is more than 20000 lines of code
- More than 1000 downloads

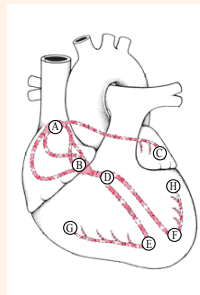
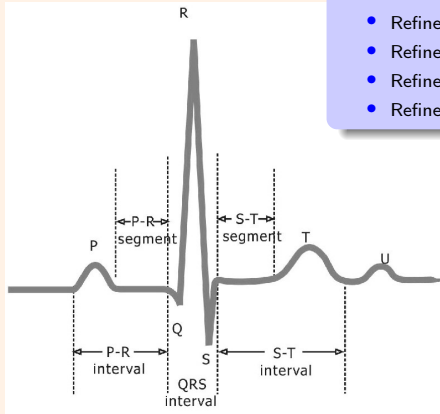
Download : <http://eb2all.loria.fr/>



Formal Development

- Abstract Model
- Refinement 1 : Introducing Steps in the Propagation
- Refinement 2 : Impulse Propagation
- Refinement 3 : Perturbation the Conduction
- Refinement 4 : Getting a Cellular Model

Electrocardiogram (ECG)



Landmarks on the Electrical Conduction of Heart

- 1 Verification
- 2 Proof Management
- 3 Development Methodology and Tools for Correct By Construction
- 4 Project Management

- **Property**

- ▶ the set of people is non empty and finite
- ▶ $people \neq \emptyset \wedge finite(people)$